

WELAB — Wii Balance Board Posture Module

Installation & Usage Manual (English)

This module turns a Nintendo Wii Balance Board (WBB) into a low-cost force plate for standing-posture assessment. One window does everything: view center-of-pressure (CoP) live, **collect** neutral vs. slouched trials with a button, **train** and compare machine-learning models, and **monitor** posture live with an alarm.

1. What you need

- A Nintendo **Wii Balance Board** with fresh batteries.
- A **Windows PC with Bluetooth** (built-in or a USB Bluetooth dongle).
- **Python 3.9+** (the same interpreter you use for WELAB).
- Python packages (only for training/monitoring): `pip install scikit-learn joblib`
- **WiimoteLib.dll** — already in your **WiiBalanceWalker** folder (used only for the Bluetooth bridge in Live mode).

Files

File	Purpose
<code>run_welab_wbb.bat</code>	double-click to launch the app
<code>wbb_gui.py</code>	the all-in-one app (connect, collect, train, monitor)
<code>wbb_core.py</code>	calibration, CoP, sway/posture features
<code>wbb_bridge.py</code>	ingest from the bridge (UDP), tare, resample
<code>wbb_record.py</code>	timed trials, CSV export, windowing
<code>wbb_dataset.py</code>	the labeled database
<code>wbb_train.py</code>	train & compare LogReg / Tree / Random Forest
<code>wbb_monitor.py</code>	live classifier + alarm
<code>WiiBoardBridge.cs</code> , <code>build_bridge.bat</code>	the Bluetooth bridge (Live mode)
<code>udp_probe.py</code>	check the bridge is streaming
<code>tests/</code>	self-tests

Keep all files in one folder.

2. First run (no hardware needed)

Double-click `run_welab_wbb.bat` . The app opens. To try it without a board:

1. Under **1. Connect**, choose **Demo (no board)**.
2. Click **Connect** — a moving CoP cursor, trail, and orange 95% ellipse appear.
3. Click **Record NEUTRAL (30s)**, then **Record SLOUCHED (30s)** a few times each (in Demo these are generated instantly so you can see the whole flow).
4. Click **Train & Compare models** — results appear and a model is saved.
5. The posture label now updates live; switch postures in Demo to see it react.

This verifies the entire pipeline before any Bluetooth setup.

If the window closes immediately, Python may not be on PATH. Open a Command Prompt in the folder and run `python wbb_gui.py` (or `py wbb_gui.py`) to see the error.

3. Pair the board (Windows, once)

1. Open the battery cover; note the small **red SYNC button**.
2. In **WiiBalanceWalker**, click "**Add/Remove Bluetooth Wii device**" to get a *Permanent PIN* (computed from your PC's Bluetooth MAC).

3. Press SYNC (blue lights blink), select **Nintendo RVL-WBC-01**, pair, and paste the Permanent PIN. After this the board reconnects via its **front button**.

If your Bluetooth MAC contains `00`, permanent pairing fails — use a USB dongle. Check with `getmac /v /fo list`.

4. Build the Bluetooth bridge (no Visual Studio)

The bridge reads the board and sends data to the app over local UDP.

- Put `WiiBoardBridge.cs`, `build_bridge.bat`, and `WiimoteLib.dll` (from your WiiBalanceWalker folder) in one folder.
- Double-click** `build_bridge.bat` → it produces `WiiBoardBridge.exe`.
- If a platform/bitness error appears, edit the `.bat` and remove `/platform:x86`.

5. Go live

- Double-click** `WiiBoardBridge.exe`, press the board's front button. Leave it open.
- (Optional) confirm streaming: `python udp_probe.py` shows `t,tr,tl,br,bl` lines.
- In the app, choose **Live board**, click **Connect**.

6. Using the app

1. Connect — pick Demo or Live, click **Connect**. Click **Zero (tare)** while standing **off** the board to remove its baseline.

Live — shows mass, CoP, Left/Right and Anterior/Posterior load bars, and live path length / velocity / 95% ellipse.

2. Collect data — type a **Subject** id, then click **Record NEUTRAL (30s)** or **Record SLOUCHED (30s)**. A banner shows which posture is recording and a countdown; each finished trial is appended to the database (`wbb_db` folder) and the counts update. Repeat many times per posture, ideally across subjects.

2. Collect data — type a **Subject** id, set the **Duration(s)** you want per trial, then click **Record NEUTRAL** or **Record SLOUCHED**. A banner shows which posture is recording and a countdown; each finished trial is appended to the database (`wbb_db`).

For each subject, first click **Record BASELINE** (default 5 s, quiet neutral stance). This one short recording defines that person's "neutral origin". It is **not** a training class — it is only used to normalize that subject's trials, the same way live monitoring captures a baseline before it starts. Record it once per subject, before their neutral/slouched trials, and keep the feet in the same place afterwards. **The baseline must be at least as long as the training Window** (a 5 s baseline needs Window $\leq$ 5 s), or the training report will warn you.

Clear dataset archives the current data (index + all raw samples) into `wbb_db/archive/<timestamp>/` and starts fresh — nothing is deleted.

3. Train & compare — set **Window(s)** (the sub-window length used for augmentation and live prediction; must be $\leq$ your trial duration **and** $\leq$ your baseline length), choose **Group**, leave **per-subject baseline normalization** checked, and pick **Baseline from**:

Group	Folds	Answers
trial	k-fold over trials	"does the signal exist, and can the model read it <i>within</i> a person?" The same people are in train and test, so it is optimistic.
subject	k-fold over people	generalization, but several people are held out per fold
L0S0	one fold per person	leave-one-subject-out: every person is held out exactly once. The standard way to report between-subject generalization.

Setting	Baseline used	Notes
auto	the subject's dedicated BASELINE recording (falls back to their neutral trials if they have none)	matches live monitoring; independent of the training labels
neutral	the mean of all that subject's neutral trials	uses much more data, but is computed from the same rows being classified

Click **Train & Compare models**. It cross-validates Logistic Regression, Decision Tree, and Random Forest, shows accuracy / precision / recall / F1 for the `slouched` class plus a **feature-importance ranking**, and saves the best model. The report also lists

each fold separately with the mean and standard deviation: a mean of 0.75 built from folds {0.75, 0.75} is a very different result from one built from {0.5, 1.0}. A large spread means the estimate is unstable — usually too few trials or subjects. With `subject` grouping, each fold names the person who was held out. Comparing `auto` vs `neutral` is a good experiment: `auto` usually gives a *lower but more honest* score, and often behaves better live.

Compare: trials-only vs LOSO runs all three designs at once and prints them side by side, which is how a paper should report this:

	folds	best	acc	F1	fold sd
trials-only k-fold	5	logreg	0.914	0.911	0.033
subject k-fold	5	rf	0.841	0.837	0.048
LOSO	12	rf	0.857	0.853	0.102

Two different questions are being answered here, so use two different rows:

- **Headline the LOSO number.** It is what this field conventionally reports, it trains on n-1 of n subjects (closest to the model you actually ship), and it gives one accuracy per person, so you can say who the model fails on.
- **Use the matched pair for the gap.** `trials-only k-fold` and `subject k-fold` use the same number of folds, so they fit on the same fraction of the data and the *only* thing that differs is whether the test people are strangers. Comparing trials-only against LOSO instead would confound the gap with training-set size.

The tool prints the matched gap, the per-subject LOSO accuracies with a mean and CI, and writes `design_comparison.png` and `loso_per_subject.png`.

Two honest caveats the report also prints. LOSO training sets overlap heavily (n-1 subjects shared between folds), so the folds are not independent and that 95% CI is optimistic. And the best model is *chosen* by its CV score, so the CV number is mildly favourable to itself. Both are exactly what Phase 2 fixes: freeze the model, then score new people. Phase 1 LOSO is the development estimate; Phase 2 is the confirmatory one.

This button does not save a model; run **Train & Compare** with the grouping you intend to report so the saved model matches your paper.

4. Live monitor — Load model file... imports any `.joblib`, including one trained in your own script rather than by this app. A bare scikit-learn estimator works: the class labels are read from the fitted model and the app's feature set is assumed. Two things cannot be inferred and are asked for, because getting them wrong makes the live output meaningless:

Asked	Why it matters
window length	the model must be fed the same number of seconds it was trained on
baseline-normalized?	if yes, each person's neutral baseline is subtracted first

To skip the questions, save the model as a dict instead of a bare estimator:

```
joblib.dump({"model": clf, "feature_names": FEATURE_NAMES,
            "labels": ["neutral", "slouched"], "positive": "slouched",
            "window_s": 10.0, "normalized": True}, "my_model.joblib")
```

A model trained on a different number of features is refused rather than used, so a stale model cannot quietly produce nonsense. **Use trained model** switches back to the one this app trains. The loaded model is summarized under the buttons.

4. Live monitor — if the model is normalized, first click **Set neutral baseline** and stand neutrally for one window (~10 s) so the app learns *your* neutral. Then click **Apply model to live**. The model is applied only when you choose (not automatically after training). When slouching is sustained, the board border flashes red and the PC beeps. Click again to stop.

Where the data lives: the `wbb_db` folder holds `dataset.csv` (a summary index — one row of features per trial) **and** a `samples/` subfolder with the full **raw time series** for every trial (`time`, `CoP x/y`, `mass`, and all four corner weights). Training and baseline normalization re-read these raw files.

5. Validate on new subjects (Phase 2) — a two-phase study is much stronger than cross-validation alone:

1. **Phase 1** — collect your training subjects in `wbb_db`, train, and let the best model be saved to `posture_model.joblib`. Choose the model using **Group = subject** (the honest estimate). Then **freeze it**: do not re-train.
2. **Phase 2** — change **Dataset folder** to a new name (e.g. `wbb_db_val`) and collect *different* people: one BASELINE each, then their neutral/slouched trials, with exactly the same protocol and settings.

Nobody from Phase 1 may appear in Phase 2. The unit of contamination is the *person*, not the recording: a fresh session from someone the model trained on still measures memory rather than generalization. Give the validation group their own ids (`v01` , `v02` , ...) so the two groups can never be confused. The app stores the training subject list inside the model and will tell you at the top of the report whether the held-out set is genuinely held out. 3. Put that folder in **Held-out folder** and click **Score frozen model on this folder**. Nothing is re-trained — the saved model is applied as-is and scored.

The report gives window-level and trial-level accuracy/precision/recall/F1, a confusion matrix, and **per-subject accuracy with a 95% confidence interval** — generalization is about people, so the number of subjects is what the interval is based on. It also compares the result against the model's own Phase 1 CV score, so you can see whether that estimate held up.

Make Phase 2 figures (PNG) writes three poster-ready plots:

File	Shows
<code>phase2_per_subject.png</code>	accuracy for each held-out person, with the mean and its 95% CI
<code>phase2_confusion.png</code>	trial-level confusion matrix on the new subjects
<code>phase2_vs_phase1.png</code>	the Phase 1 CV estimate next to the Phase 2 reality

The moment you look at Phase 2 and then change the model, it stops being a held-out test and becomes a second training set. Decide everything in Phase 1.

Because the model is baseline-normalized, each Phase 2 subject needs their own BASELINE recording — that 10 s calibration is part of the method being tested, so report the system as "classifies a new person **after a 10 s neutral calibration**". To measure what the calibration is worth, freeze a second model with normalization switched off and score the same folder with it.

The same check is available from the command line:

```
python wbb_validate.py --db wbb_db_val --model posture_model.joblib
```

7. Data-collection tips

- Tare with the board empty before each session.
- Keep **foot position consistent** on the board — CoP position is the strongest slouch cue and shifts if the feet move.
- Aim for ~15–20 trials per posture, and several subjects, for a robust model.

8. Advanced: command line (optional)

The same actions exist as scripts, e.g.:

```
python collect_posture.py --label neutral --subject S01 --db wbb_db
python collect_posture.py --label slouched --subject S01 --db wbb_db
python wbb_train.py --db wbb_db --window 10 --hop 5 --out posture_model.joblib
python wbb_gui.py --port 8674 --model posture_model.joblib
```

9. Troubleshooting

- **Window closes instantly** — run `python wbb_gui.py` in a Command Prompt to see why.
- `build_bridge.bat` : **"csc.exe not found"** — install **.NET Framework 4.8**.
- **No live data** — confirm `WiiBoardBridge.exe` is running and the board is on; the app's port must match the bridge (default 8674).
- **Training says "need ≥2 classes"** — collect more trials of each posture.

10. Notes and limitations

- A single board gives CoP magnitude with placement sensitivity; keep feet consistent. CoP origin is the board center (cm).
- The WBB samples at a variable rate (~100 Hz); data is resampled to a fixed rate.
- Models train on windows (default 10 s) and the monitor predicts over the same window length. `trial` grouping keeps each trial in one fold (no window leakage); `subject` grouping estimates generalization to new people (usually a lower but more honest score).

- Features include CoP position, sway amplitude/area/velocity, load asymmetry, and **frequency-domain sway** (mean & median sway frequency for ML and AP). Body mass is excluded (not posture).
- **Raw data is required for training:** windowed augmentation and baseline normalization re-read each trial's raw CoP series from `samples/`. Keep that folder; the summary `dataset.csv` alone is not enough to (re)train.
- Validate CoP sign/scale against an established tool (CU BrainBLoX) and the calibration literature (Clark et al. 2010; Leach et al. 2014) before reporting.